PROJECT REPORT

MYRA : My RAG Assistant

A Retrieval-Augmented Generation (RAG) System for Academic NLP Learning

---

1. Title Page

- **Project Title:** MYRA : My RAG Assistant
- **Domain:** Natural Language Processing & Generative AI
- **Institution:** Malaviya National Institute of Technology (MNIT), Jaipur
- **Department:** Artificial Intelligence and Data Engineering
- **Team Members:**
  - Ankit Kumar (2023UAI1999)
  - Shivam Kuntal (2023UAI1888)
  - Gatik Singh Kanyal (2023UAI1058)
- **Academic Session:** 2025-2026

2. Abstract

Standard Large Language Models (LLMs) often suffer from technical hallucinations when queried about niche academic content. This report details the implementation of **MYRA (My RAG Assistant)**, a modular Retrieval-Augmented Generation (RAG) system. The system utilizes a two-stage retrieval pipeline—combining FAISS-based vector search with Cross-Encoder reranking—to ground LLM responses in verified NLP textbooks. Key features include a semantic conversation memory, a failover LLM router between Groq and Gemini, and an automated grounding evaluation loop.

---

3. Introduction

In the rapidly evolving field of Natural Language Processing, students require precise, technically accurate information. While LLMs like LLaMA and Gemini are powerful, their training data is static and prone to "hallucinations"—generating plausible but incorrect technical details. RITA (Retrieval-based Intelligent Teaching Assistant) addresses this by implementing a RAG framework that retrieves relevant context from academic sources before generating an answer.

---

4. Problem Statement

Standard LLMs encounter the following issues in academic settings:

- **Technical Inaccuracy:** Hallucinating mathematical derivations or architectural details.
- **Lack of Traceability:** Inability to provide specific page-level citations for claims.
- **Generic Responses:** Providing surface-level advice rather than the depth found in core textbooks.

---

5. Objectives

- **Grounding:** Minimize hallucinations by grounding responses in "Natural Language Processing: Neural Networks and Large Language Models" by Tong Xiao and Jingbo Zhu.
- **Resilience:** Implement a multi-tier fallback architecture to ensure high system availability.
- **Validation:** Verify answer integrity using weighted cosine similarity scoring.
- **UX:** Provide a customizable UI for varying explanation depths (ELI5 to Professional).

---

6. Literature Review

The project builds upon the **RAG (Retrieval-Augmented Generation)** paradigm introduced by Lewis et al. It incorporates modern advancements in **Bi-Encoder** retrieval for speed and **Cross-Encoder** reranking for precision. We also utilize the "Lost in the Middle" research findings to limit context selection to the top 4 most relevant chunks, optimizing the LLM's attention.

---

7. Technologies Used

Category	**Tool/Library**
Language	Python 3.10
LLM Orchestration	LangChain
Vector Database	FAISS (Facebook AI Similarity Search)

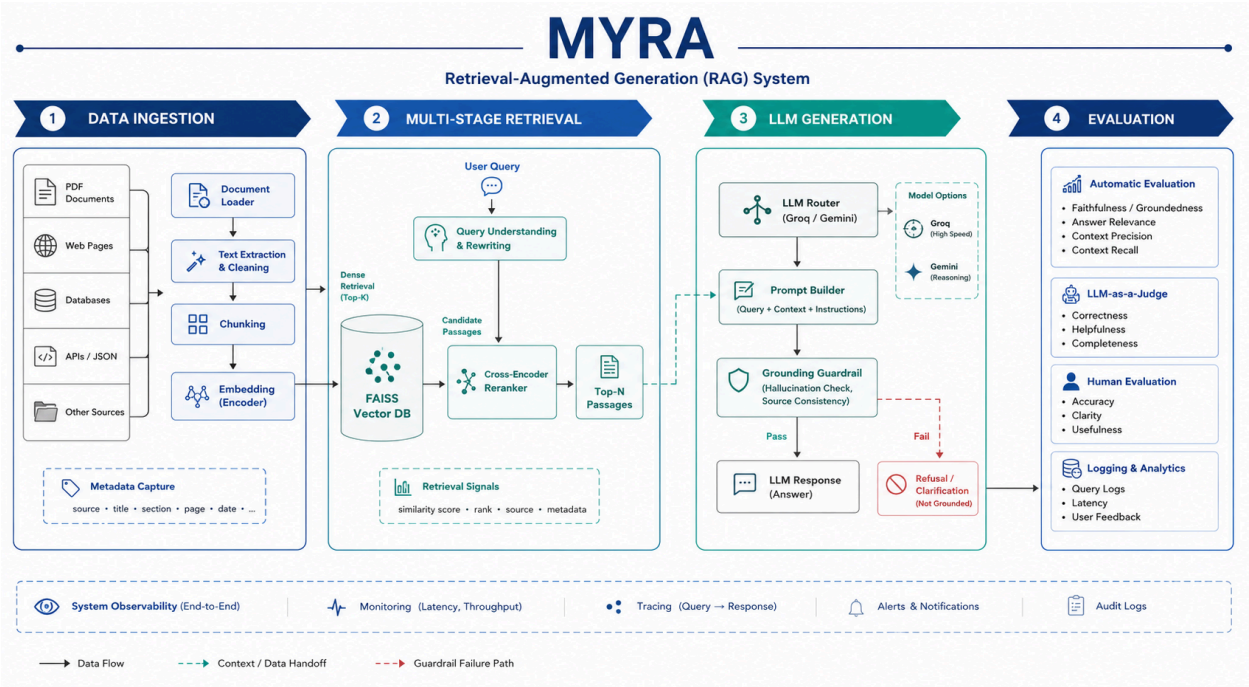
Embeddings	all-MiniLM-L6-v2 (Sentence-Transformers)
Reranker	ms-marco-MiniLM-L-6-v2 (Cross-Encoder)
Frontend	Streamlit
LLM Providers	Groq (LLaMA 3.3), Gemini (3 Flash)

---

8. System Architecture

The architecture is divided into four distinct modules:

1. **Ingestion:** PDF parsing and vector indexing.
2. **Retrieval:** Two-stage similarity search.
3. **Generation:** Prompt synthesis and LLM routing.
4. **Evaluation:** Grounding verification.



9. Methodology

We adopted a modular engineering approach:

- **Data Prep:** Extracting text via PyPDFLoader.
- **Storage:** Persisting vectors locally in a FAISS index.
- **Execution:** A query-time pipeline that first retrieves context, then builds a prompt, then generates and evaluates the answer.

10. Data Flow

1. **User Input** → 2. **Query Normalization** → 3. **Vector Search** → 4. **Reranking** → 5. **Memory Retrieval** → 6. **Prompt Building** → 7. **LLM Generation** → 8. **Grounding Evaluation** → 9. **UI Display**.

11. NLP Pipeline

The NLP pipeline focuses on text normalization and segmentation.

- **Cleaning:** Removing excessive whitespace and filtering chunks under 50 characters to eliminate noise (page numbers, headers).
- **Segmentation:** Utilizing RecursiveCharacterTextSplitter to maintain paragraph structure.

12. Embedding and Retrieval Mechanism

- **Embedding:** Text is converted into 384-dimensional dense vectors using all-MiniLM-L6-v2.
- **Bi-Encoder Retrieval:** Initial fast search retrieves top 10 candidates based on cosine similarity.
- **Cross-Encoder Reranking:** The 10 candidates are paired with the query and scored by ms-marco-MiniLM-L-6-v2 to select the top 4 chunks.

13. Vector Database: FAISS

We chose **FAISS (CPU)** for local persistence. FAISS enables efficient nearest-neighbor search. The index is stored in the faiss_index/ directory, allowing the system to boot without re-processing PDFs.

14. Prompt Engineering Approach

Prompts are constructed dynamically in prompt_builder.py:

- **System Instructions:** Explicit constraints to use only provided context.
- **Style Injection:** Conditional instructions based on user sidebar selection (e.g., "Use analogies" for 'analogy' tone).
- **Citations:** The LLM is instructed to use [Doc X] markers for every claim.

---

15. Streamlit Frontend

The UI provides a ChatGPT-like experience with:

- **Sidebar Controls:** For adjusting Tone, Depth, Language, and Format.
- **Multi-Chat Support:** Each session maintains its own ConversationMemory.
- **Metadata Toggles:** Users can view the internal confidence score and source files for each answer.

---

16. Implementation Details

- **Environment:** Managed via .env for API keys and requirements.txt for dependencies.
- **Paths:** Use of os.path.abspath to ensure the system runs consistently across different directories.

---

17. Important Code Modules

- **src/retriever.py:** Implements the core retrieve() and rerank() logic.
- **src/llm.py:** Manages the LLM class with Groq/Gemini failover.
- **evaluation/evaluate.py:** Computes the grounding score.

Python

```python
# Sample Reranking Logic from retriever.py
def rerank(query, docs, top_k=4):
    pairs = [(query, d.page_content) for d in docs]
    scores = reranker.predict(pairs)
```

```
ranked = sorted(zip(docs, scores), key=lambda x: x[1], reverse=True)
return [doc for doc, _ in ranked[:top_k]]
```

---

18. Challenges Faced

- **Equation Fragmentation:** Fixed by increasing chunk_overlap to 150.
- **Retrieval Noise:** Mitigated by filtering chunks < 50 characters.
- **Hallucination Control:** Solved by integrating a hard 0.4 grounding threshold guardrail.

---

6. Role of LLM Assistance

Artificial Intelligence served as a collaborative partner in several key aspects of the project:

- **Architectural Brainstorming:** Aiding in the design of the multi-tier fallback logic within the `llm.py` module.
- **Logic Refinement:** Providing assistance with the implementation of the Cross-Encoder reranking logic, located in `retriever.py`.
- **Prompt Engineering:** Generating dynamic prompt templates for the `prompt_builder.py` module to effectively handle "Hinglish" and "ELI5" (Explain Like I'm 5) response styles.
- **Code Generation :** The core functionality of the code within this project was primarily developed with the assistance of Artificial Intelligence. The AI was directed and guided through precise prompts that outlined the necessary tasks and specified project requirements.
- **Error Resolution and Key Learnings:** Developing this project provided a hands-on learning experience with Retrieval-Augmented Generation (RAG). Errors encountered during development were resolved, leading to a deeper understanding of the RAG concept and ultimately improving the project's functionality.

---

23. Limitations

- **Linear Memory Scaling:** Embedding-based memory may slow down in very long

conversations.
- **Lack of Hybrid Search:** The current implementation does not combine keyword-based search with semantic search, which is a standard industry practice for improving retrieval robustness.
- **Single-Metric Reliance:** The evaluation loop primarily uses weighted cosine similarity to calculate confidence
- **Dense-Only Retrieval:** Lacks keyword-based search (BM25), which may miss specific acronyms.

---

24. Future Enhancements

- **Hybrid Search:** Integration of BM25 with FAISS for better keyword matching.
- **Advanced Evaluation:** Moving to RAGAS framework for metrics like faithfulness and relevancy.
- **Agentic Search:** Allowing the AI to "re-search" if initial results are poor.

---

25. Conclusion

The RITA project successfully demonstrates that a modular RAG architecture can effectively mitigate LLM hallucinations in technical education. By combining multi-stage retrieval with rigorous evaluation guardrails, we provide a trustworthy tool for NLP students.

---

26. References

1. Xiao, T., & Zhu, J. (2026). *Natural Language Processing: Neural Networks and Large Language Models*.
2. LangChain Community. (2024). *RAG Best Practices*.
3. Meta AI & Google. *Documentation for LLaMA 3.3 and Gemini 3*.